



Từ Chatbot Đến Agentic Agent

AICB-P1 · Ngày 3 · Design Pattern ReAct

Tên Giảng Viên

VinUniversity · Phase 1 · Tuần 1 · 17/03/2026

HÃY SUY NGHĨ...

*“ChatGPT là chatbot hay agent?
Siri thì sao? Cursor IDE thì sao?”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. 3 Kiểu Hệ Thống AI
2. Agentic Fit Framework
3. Kiến Trúc Agent
4. ReAct Pattern
5. Agent Loop: Code Anatomy
6. Live Demo & Debug
7. Eval & Telemetry
8. Chatbot vs Agent
9. Lab 3

- Phân biệt được **rule-based bot**, **LLM chatbot**, và **agent**
- Dùng **Agentic Fit** để biết khi nào nên nâng từ chatbot lên agent
- Hiểu và giải thích được vòng lặp **ReAct: Thought → Action → Observation**
- Build được **ReAct agent đầu tiên** với tools, system prompt, và safeguard cơ bản
- Phân biệt **text-ReAct vs native tool calling**; biết các **failure mode** và đo bằng **telemetry**

Chatbot baseline + ReAct agent (native tool calling) cho cùng bài toán, chạy được end-to-end kèm telemetry.

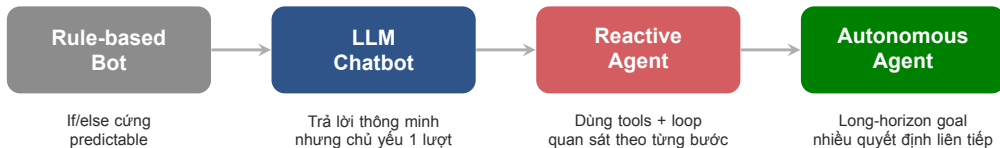
- **Group report:** so sánh chatbot vs agent trên bộ scenario, kèm **1 trace thành công + 1 trace lỗi**
- **Individual report:** đóng góp kỹ thuật + 1 case debug đọc từ log
- Chấm theo rubric **100 điểm** (group 60 / individual 40) — xem slide rubric ở phần Lab

3 Kiểu Hệ Thống AI

Từ bot có rule đến agent có khả năng lập kế hoạch và dùng công cụ

01

Khả năng thích nghi, tool use, memory, risk tăng dần



Chatbot phản ứng (reactive: trả lời khi được hỏi rồi dừng); **agent hướng mục tiêu** (goal-driven: tự quyết định, dùng tool, lặp đến khi đạt goal). Không phải mọi thứ dùng LLM đều là agent — agent chỉ xuất hiện khi hệ thống phải quyết định, hành động, quan sát kết quả, rồi lặp lại.

Tiêu chí	Rule-based Bot	LLM Chatbot	Agent
Cách xử lý	If/else cố định	Sinh câu trả lời tốt theo context	Plan → act → observe → adapt
Flexibility	Thấp	Trung bình	Cao
Memory	Gần như không có	Ngắn hạn trong con-text	Ngắn hạn + có thể thêm long-term mem-ory
Tool use	Hard-coded	Có thể gọi tool theo chỉ định	Chủ động chọn tool theo bước tiếp theo
Cost	Thấp nhất	Trung bình	Cao hơn do loop và nhiều calls
Risk	Logic dễ kiểm soát	Hallucination / format drift	Hallucination + tool misuse + loop
Ví dụ phù hợp	Menu IVR (tổng đài), form validation	FAQ, support cơ bản	Booking, research, coding assistant

So sánh trực quan để chọn đúng mức độ phức tạp

Bài toán: “Tìm vé HAN → HCM dưới 2 triệu, rồi gợi ý mang gì nếu trời mưa.”

Bot có rule

- Trả menu lựa chọn cố định
- Không search được dữ liệu mới
- Không tổng hợp nhiều điều kiện

LLM chatbot

- Viết câu trả lời mượt
- Nhưng không tự truy vấn giá vé thật

Reactive agent

- Tách goal thành 2 việc: tìm vé + check thời tiết
- Gọi từng tool theo bước
- So sánh kết quả rồi trả lời gộp

Lưu ý: Nếu bài toán không cần dữ liệu mới, nhiều bước, hay quyết định động, agent thường là overkill.

Agentic Fit Framework

4 tiêu chí để biết bài toán có thật sự cần agent hay không

02

1. Multi-step Reasoning

Bài toán có cần chia thành nhiều bước phụ thuộc nhau không?

2. Tool Interaction

Hệ thống có cần gọi search, API, database, calculator, browser, file system...?

3. Dynamic Decision

Mỗi bước tiếp theo có phụ thuộc vào kết quả vừa quan sát không?

4. Long Horizon

Hệ thống có phải giữ mục tiêu xuyên suốt qua nhiều vòng lặp hoặc nhiều state không?

Nếu đa số tiêu chí chỉ ở mức 1–2/5, hãy bắt đầu bằng chatbot hoặc workflow đơn giản.

Use case	Reasoning	Tool use	Dynamic sion	deci-	Tổng
FAQ nội bộ HR	1	1	1		3
Tóm tắt hợp đồng và high-light risk	3	2	2		7
Booking assistant du lịch	4	5	4		13
Research agent tìm đối thủ cạnh tranh	4	4	4		12
Code assistant có test & fix loop	5	5	4		14

Gợi ý đọc điểm: 0–5 = chatbot/rule đủ 6–10 = augmented chatbot 11+ = agent đáng thử

Chấm nhanh theo thang 1–5 cho từng tiêu chí

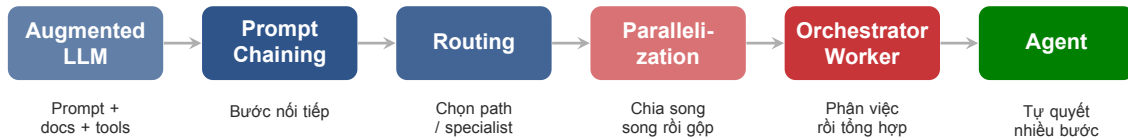
- ☐ **Bài toán 1 bước:** hỏi đáp, tra FAQ, phân loại cơ bản
- ☐ **Không có tool nào để gọi:** agent chỉ “suy nghĩ” nhưng không hành động được
- ☐ **Mọi thứ phải 100% deterministic:** mỗi sai sót đều rất đắt
- ☐ **Chi phí latency không chấp nhận được:** loop 3–5 bước là đã quá chậm
- ☒ **Nguyên tắc:** luôn benchmark rule / workflow / chatbot trước khi mở agent loop

Customer FAQ

- Câu hỏi lặp lại, intent khá ổn định
- Chủ yếu retrieve policy rồi trả lời
- Có thể thêm RAG (Retrieval-Augmented Generation) nhưng chưa cần autonomy
- **Best fit:** chatbot có retrieval

Booking Assistant

- Nhiều ràng buộc: thời gian, ngân sách, preference
- Phải search, so sánh, hỏi lại, rồi chốt phương án
- Bước sau phụ thuộc kết quả bước trước
- **Best fit:** reactive agent có tool use



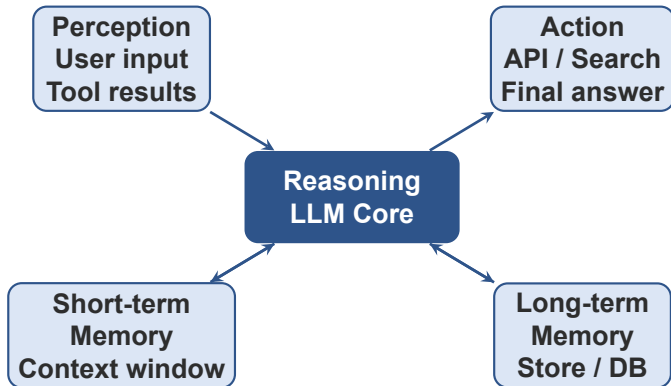
Bắt đầu từ cấu trúc đơn giản nhất đủ dùng. *Workflow* (đường đi cố định): prompt chaining → routing → parallelization → orchestrator–worker → evaluator–optimizer. *Agent* là khi model tự định hướng quy trình — mạnh nhất nhưng đắt nhất về cost, eval, guardrails.

Kiến Trúc Agent

Perception, reasoning, action, memory và luồng thông tin giữa các khối

03

Input từ môi trường



State và memory giúp agent không "mất mạch"

- **Perception:** agent nhận text, tool output, feedback
- **Reasoning:** phân tích trạng thái và chọn bước tiếp theo
- **Action:** gọi tool hoặc trả lời user
- **Memory:** giữ goal, facts, và intermediate results

4 khối kiến trúc thường kéo theo 4 nhóm cost chính: token, storage, API, và latency.

Short-term memory

- Nằm trong context window
- Dùng cho task hiện tại
- Rẻ để implement, nhưng dễ đầy

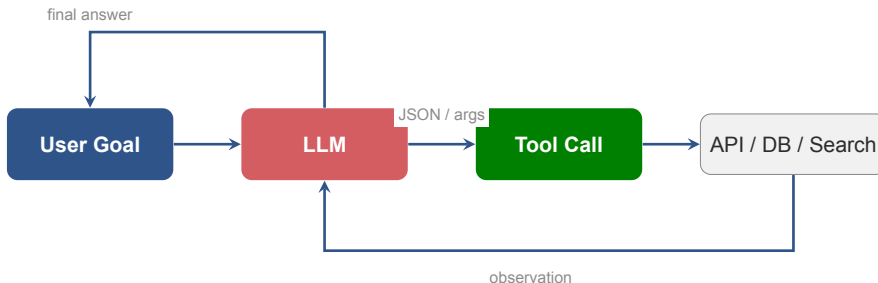
Phù hợp khi

- Cuộc hội thoại ngắn
- Goal chỉ kéo dài vài bước

Long-term memory

- Lưu facts, preferences, hay state ngoài context
- Có thể là DB, vector store, key-value store
- Cần retrieval strategy và permission model

Lưu ý: Không phải thêm memory là agent giỏi hơn. Memory chỉ có ích khi chiến lược đọc/ghi và quyền truy cập được thiết kế rõ. (*Hiện thực long-term memory với vector store: Ngày 7.*)



- Tool definitions phải rõ input / output / error mode
- Agent mạnh lên nhờ tool, nhưng cũng dễ fail hơn vì external dependency
- Hôm nay: tool calling như một **khái niệm** trong vòng ReAct. **Ngày 4**: tool như một **contract** (JSON Schema, mô tả tool, parallel calls)

ReAct Pattern

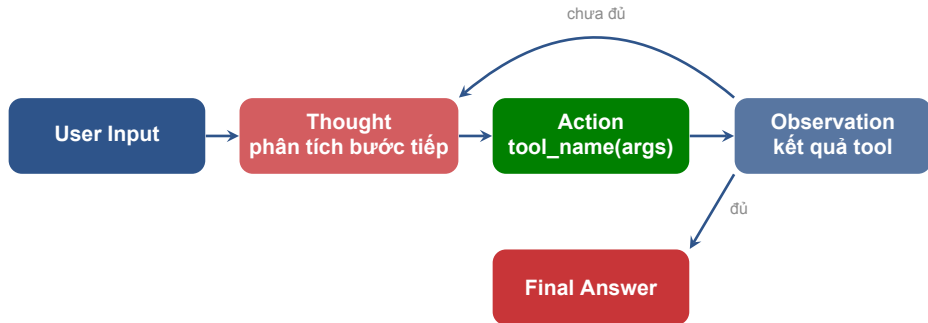
Reasoning + Acting: cách đơn giản nhất để biến LLM thành agent có thể debug được

04

ReAct = Reasoning + Acting

ReAct là pattern kết hợp **suy luận theo từng bước** với **gọi công cụ và quan sát kết quả**. Thay vì trả lời ngay, agent sẽ lặp qua các bước:

- **Thought**: mình đang thiếu gì, nên làm gì tiếp?
- **Action**: gọi tool nào, với tham số nào?
- **Observation**: kết quả trả về là gì?
- Lặp lại đến khi đủ thông tin để trả lời hoặc gặp điều kiện dừng



ReAct mạnh vì trace lý do hành động được bộc lộ ra ngoài, giúp con người debug và can thiệp dễ hơn so với chỉ nhìn final answer.

Thought 1: Tôi cần tìm chuyến bay sáng mai từ HAN tới HCM dưới 2 triệu.

Action 1: `search_flights(origin="HAN", destination="SGN", date="2026-03-18", max_price=2000000)`

Observation 1: Có 2 lựa chọn phù hợp: VietJet 06:10 giá 1.75M, Vietnam Airlines 08:20 giá 1.95M.

Thought 2: User cũng hỏi về trang phục nếu trời mưa. Tôi cần check thời tiết tại HCM.

Action 2: `get_weather(city="Ho Chi Minh City", date="2026-03-18")`

Mỗi Thought nên làm rõ agent còn thiếu gì; mỗi Action phải chỉ ra tool và args đủ cụ thể để kiểm tra.

Observation 2: Nhiệt độ 27–32C, xác suất mưa 70%.

Thought 3: Tôi đã có cả lựa chọn chuyến bay và dữ liệu thời tiết. Có thể kết luận.

Final Answer: Gợi ý chuyến 06:10 giá 1.75M; mang áo mỏng, giày dễ khô, ô gấp hoặc áo mưa nhẹ.

Điểm mạnh không chỉ là answer tốt hơn, mà là con người nhìn được agent đã hành động dựa trên quan sát nào.

Ưu điểm

- Dễ đọc trace và debug
- Tự quyết được bước tiếp theo từ observation
- Phù hợp các bài toán search / booking / investigation / coding
- Có thể cài safeguard ở từng vòng lặp

Giới hạn

- Tốn nhiều token và latency hơn chatbot
- Dễ loop hoặc gọi sai tool
- Cần eval theo trace, không chỉ final answer
- Không phù hợp bài toán đơn giản hoặc cần deterministic tuyệt đối

Lưu ý: ReAct dễ bắt đầu nhất, nhưng khi hệ thống nhiều nhánh hơn, nên chuyển sang graph/state machine rõ ràng.

Quan trọng — ReAct là **mô hình tư duy** (Thought → Action → Observation). *Cách hiện thực* bước Action có 2 kiểu — và production 2026 mặc định chọn kiểu thứ hai.

Text-based ReAct (paper gốc, 2022)

- Model in ra **text**: Action: `get_weather("HCM")`
- **Code của bạn** tự parse (regex / JSON)
- Chạy trên mọi text model (kể cả local nhỏ)
- Dễ vỡ: lệch format, dính markdown fences, args bị

Native function/tool calling (2024–2026)

- Model in ra **object có cấu trúc** (đã được post-train)
- **API** trả args đã theo schema; *strict mode* đảm bảo đúng schema
- Tin cậy hơn, ít parse error
- Cần model / endpoint hỗ trợ tool calling

Học ReAct để hiểu *vòng lặp*; hiện thực bằng *native tool calling* cho production. Ngày 4 đi sâu tool như một *contract* (JSON Schema, mô tả tool, parallel calls).

```
# --- Text-ReAct: ban tu parse (de vo) ---
text = "Action: get_weather(city='HCM')"
m = re.match(r"Action:\s*(\w+)\s*((.*)\s)", text) # brittle
name, args = m.group(1), m.group(2)

# --- Native tool calling: API tra ve object (tin cay hon) ---
call = resp.choices[0].message.tool_calls[0]
name = call.function.name # "get_weather"
args = json.loads(call.function.arguments) # da theo schema
```

Lưu ý: Instructor guide gọi bước parse này là “the most common bottleneck”. Lab cho bạn trải nghiệm cả hai: text-ReAct cho local model, native tool calling cho OpenAI.

Agent Loop: Code Anatomy

Từ prompt, tool registry, đến loop control và framework hóa

```
messages = [user_message(user_goal)] # seed loop voi input của user

for step in range(MAX_ITERATIONS):
    output = call_model(
        system=SYSTEM_PROMPT,
        messages=messages,
        tools=TOOLS,
    )
    if output.type == "final_answer":
        return output.content

    result = run_tool(output.name, output.args)
    messages += [
        output.as_message(),
        tool_message(output.name, result),
    ]

return "Stopped: max iterations reached"
```

```
SYSTEM_PROMPT = """
```

```
You are a travel planning agent.
```

```
Your job:
```

- Break the user goal into smaller steps*
- Use tools when fresh information is required*
- Think briefly, then choose the best next action*
- Stop when you have enough evidence to answer*

```
Rules:
```

- Never invent tool results*
- If a tool fails, explain the failure and try a fallback*
- Keep internal thoughts short and actionable*
- Output either a tool call or a final answer*

```
"""
```

```
TOOLS = {  
  "search_flights": {  
    "description": "Tim chuyen bay theo route/ngay/ngan sach",  
    "args": ["origin", "destination", "date", "max_price"],  
  },  
  "get_weather": {  
    "description": "Thoi tiet theo thanh pho/ngay",  
    "args": ["city", "date"],  
  },  
  "recommend_outfit": {  
    "description": "Goi y trang phuc theo nhiet do + xac suat mua",  
    "args": ["temp_high_c", "rain_probability"],  
  },  
}
```

Cần guardrails gì?

- Giới hạn số vòng lặp
- Timeout cho từng tool
- Budget token / cost trần
- Retry có kiểm soát
- Fallback sang human hoặc chatbot

Dấu hiệu loop

- lặp lại cùng một tool call
- hỏi lại thông tin đã có
- reasoning không tiến thêm
- observation không thay đổi nhưng vẫn tiếp tục

Khi output không tiến triển, cùng một tool bị gọi lặp lại, hoặc observation không đổi mà agent vẫn tiếp tục, cần dừng loop và fallback.

State-graph — Mô tả agent như **nodes** (bước) + **edges** (luồng) + **state** (dữ liệu mang theo), thay cho một vòng for ad-hoc.

- ReAct loop bằng tay: **tốt nhất để học bản chất** và cho agent ít nhánh
- Khi workflow **nhiều nhánh**, cần **persist state** hoặc **conditional routing** rõ ràng → chuyển sang state-graph (vd **LangGraph**)
- Đi sâu vào graph/state machine: **Ngày 9** (multi-agent) và **P2-T3 Ngày 8**

LangGraph đạt v1.0 (10/2025). `create_react_agent` (prebuilt) đã deprecated → dùng `langchain.agents.create_agent`.

Live Demo & Debug

Build agent tra cứu thời tiết và gợi ý trang phục ngay trên lớp

06

1. Định nghĩa 2 tools: `get_weather` và `recommend_outfit`
2. Viết system prompt: agent chỉ được kết luận khi đã có dữ liệu thời tiết
3. Chạy loop và đọc trace Thought / Action / Observation
4. Cố tình tạo lỗi: tool timeout hoặc agent chọn sai outfit
5. Debug: sửa prompt, sửa tool description, hoặc thêm safeguard

Cho học viên thấy agent fail ở đâu và vì sao trace lại quan trọng hơn một final answer “trông có vẻ đúng”.

```
def get_weather(city: str, date: str) -> dict:
    return {
        "city": city,
        "date": date,
        "temp_high_c": 32,          # field khop voi recommend_outfit
        "rain_probability": 0.7,
    }

def recommend_outfit(temp_high_c: int, rain_probability: float) -> str:
    if rain_probability > 0.5:
        return "Ao mong, giay de kho, mang theo o gap."
    if temp_high_c > 30:
        return "Ao nhe, thoang, uu tien vai cotton."
    return "Trang phuc thoai mai, co the mang ao khoac nhe."
```

Nhìn vào trace trước

- Thought có đúng mục tiêu không?
- Agent chọn đúng tool chưa?
- Args truyền vào có hợp lệ không?
- Observation có bị thiếu field quan trọng không?

4 nơi thường phải sửa

- Tool description quá mơ hồ
- System prompt thiếu rule dừng
- Không có safeguard cho retry / loop
- Evaluation chỉ chấm final answer, không chấm trace

Lưu ý: Agent debugging gần với debugging distributed system hơn là chỉ prompt tuning. Ta phải nhìn cả model, tool, state, và orchestration.

Lỗi	Triệu chứng	Cách xử lý
Parse error	Model in Action sai format, regex/J-SON fail	Native tool calling; strict schema; retry có hướng dẫn
Hallucinated tool	Gọi tool không tồn tại	Validate tên tool theo registry; báo lỗi rõ
Hallucinated args	Args sai / thiếu / bịa	JSON Schema + strict mode; default an toàn
Empty observation	Tool trả “không có dữ liệu”	Cho agent thử tool khác / hỏi lại / dừng
Timeout / loop	Lặp tool, reasoning không tiến	max_iterations; phát hiện lặp; fall-back human

Đây chính là taxonomy bạn sẽ gặp & phải phân tích ở **phase failure-analysis** của Lab 3 — phần được chấm điểm nặng nhất.

Eval & Telemetry

Đo bằng trace, không chỉ final answer: tokens, latency, cost, loop count

07

Step	Action	Tokens	Latency	Pass?
1	check_stock("iPhone")	180	0.6s	✓
2	get_discount("WINNER")	120	0.4s	✓
3	calc_shipping(2, "Hà Nội")	140	0.5s	✓
4	final_answer	90	0.3s	✓
Tổng	4 bước (loop count = 3)	530	1.8s	est. cost

Lab log mỗi bước ra JSON: token, latency, loop count, error code. Bạn so sánh **agent v1 vs v2** bằng **số liệu**, không phải cảm tính. (Lab dùng total latency, không phải TTFT, vì call là non-streaming.)

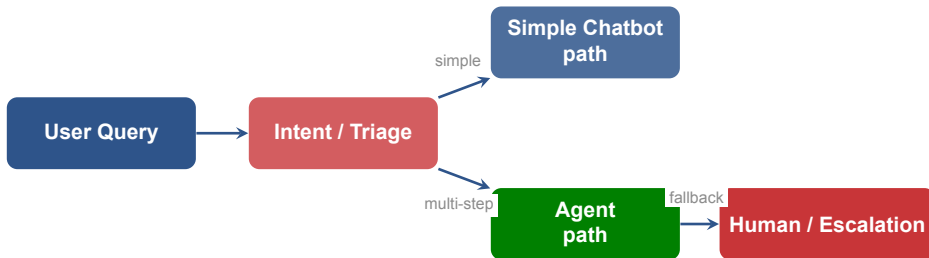
Chatbot vs Agent

Khi nào mỗi loại thắng và tại sao hybrid pattern thường thực dụng nhất

08

Khía cạnh	Chatbot thắng	Agent thắng
Tác vụ	FAQ, support đơn giản, nội dung 1 lượt	Booking, research, coding, data analysis nhiều bước
Tốc độ	Nhanh, ít round-trip	Chậm hơn do loop và tool calls
Cost	Thấp hơn, predictable hơn	Cao hơn nhưng đổi lại xử lý được bài toán khó hơn
Kiểm soát	Dễ hơn, ít state	Khó hơn vì cần orchestration và eval theo trace
UX	Phản hồi nhanh, đơn giản	Tạo cảm giác “làm việc giúp bạn” nếu làm tốt

Bắt đầu bằng chatbot là lựa chọn mặc định tốt



Không cần chọn một phe. Thiết kế tốt thường là: triage nhanh, câu đơn giản đi chatbot path, câu phức tạp mới mở agent loop.

Thực Hành

Lab 3: Chatbot vs Agent — Hands-on Comparison

09

Trên lớp (lecture)

- Domain: **chuyến bay + thời tiết**
- `search_flights`, `get_weather`,
`recommend_outfit`

Trong lab (hands-on)

- Domain: **e-commerce**
- `check_stock`, `get_discount`,
`calc_shipping`

Vòng lặp ReAct giống hệt — chỉ đổi bộ tools. Pattern **độc lập với domain**: đó chính là điều cần nhớ.

1. `cp .env.example .env`, chọn provider (OpenAI / local llama.cpp)
2. Chạy **chatbot baseline** (`chatbot.py`) trên bộ scenario e-commerce
3. Hiện thực **ReAct loop** trong `agent.py` với 2–3 tools (`check_stock`, `get_discount`, `calc_shipping`)
4. **Failure analysis**: đọc log JSON, tìm parse error / hallucinated tool / loop
5. Sửa prompt + tool description (**v1** → **v2**), rồi **group eval** so sánh bằng số liệu

Nhờ AI generate scaffolding, nhưng nhóm tự sửa system prompt, tool description, và điều kiện dừng. Xem VIBE-CODING.md.

Mục tiêu: Build chatbot baseline rồi nâng cấp thành ReAct agent (native tool calling) cho cùng use case e-commerce, chạy được kèm telemetry

Deliverable: **Nộp:** group report + individual report (rubric 100đ) + 1 trace thành công & 1 trace lỗi

Bonus: provider thứ 2 (Gemini), human-escalation, hoặc metrics nâng cao

Thời gian: 150 phút

Hạng mục	Điểm
<i>Group — 60 điểm</i>	
Chatbot baseline + ReAct agent chạy được (2+ tools)	16
Cải tiến v1 → v2 từ failure trace	12
Trace quality (thành công + lỗi)	12
Eval & so sánh chatbot vs agent (data-driven)	12
Code quality + telemetry tích hợp	8
<i>Individual — 40 điểm</i>	
Đóng góp kỹ thuật (modules / tools / tests)	15
Case debug đọc từ log (hallucination / loop / parse)	15
Reflection: chatbot vs agent	10

Sinh viên phải thấy rubric trước khi làm

- 1 Agent không phải “chatbot thông minh hơn”; agent = **LLM + reasoning + tools + memory/s-tate**
- 2 ReAct = **mô hình tư duy** (Thought/Action/Observation); production hiện thực bằng **native tool calling**
- 3 Chỉ dùng agent khi bài toán có **multi-step reasoning, tool use, dynamic decisions, long horizon**
- 4 Đánh giá agent **theo trace** (token, latency, loop, error code), không chỉ final answer
- 5 ReAct quay lại xuyên suốt khóa: **Ngày 4** (tool contract), **Ngày 9** (multi-agent), **P2-T3** (Reflexion, LangGraph)

Prompt Engineering & Tool Calling

“Ngày mai ta đi sâu hơn vào cách viết system prompt production-grade và mô tả tools để agent dùng đúng ý.”

- Đọc lại trace lab hôm nay và tìm 1 chỗ agent ra quyết định chưa tối ưu
- Thử viết lại tool description theo hướng rõ input, output, và failure mode hơn

- 1 Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629, Oct 2022; ICLR 2023.
- 2 Anthropic. *Building Effective Agents*. anthropic.com/research/building-effective-agents (19/12/2024)
- 3 OpenAI. *Function Calling & Structured Outputs*. platform.openai.com/docs/guides/function-calling
- 4 LangChain. *LangGraph v1.0 & create_agent*. docs.langchain.com — `create_react_agent` đã deprecated

Hỏi & Đáp

Use case nào trong công việc của bạn chỉ cần chat-bot, và use case nào thực sự cần agent loop?



Cảm ơn!

Email: lecturer@vinuni.edu.vn

Slides & tài liệu: github.com/VinUni-AI20k

Lab 3 repo: github.com/VinUni-AI20k/Day-3-Lab-Chatbot-vs-react-agent